## When to use)

- problems containing array or string
- asking for / scan through subarray or substring
- when the subarray must satisfy some condition (shortest / longest / min / max)
- improving time complexity from $O(n^2) \rightarrow O(n)$

## Types)

- variable or dynamic length | size of window changes
- fixed length | size of window is fixed (maybe of size k)

## What it is)

- a sliding window is when you have two pointers, say i and j or left and right. You move the j pointer as far as you can till the condition is no longer valid, then move i closer to j until the condition is valid again. At every iteration keep track of the min/max length of subarray to compare for result.

## Why use it)

- without sliding window, we would need a doubley nested for loop $\rightarrow O(n^2)$. This technique improves the time complexity to $O(n)$

### Dynamic

a b a c b a

i j

seen = { a : 2 , b : 1 , c : 3 }

### Fixed

1 3 -2 5 9 2 6 -1

i j

sum : 17

```python
# Dynamic Problem
def lengthOfLongestSubstring(s: str) -> int:
    # use dynamic since the substring can change length based on number of non-duplicates

    st = set() # make a set to ensure no duplicate characters
    i = 0 # initialize pointer
    longest = 0 # save longest length for return
    if len(s) == 0: # error handling
        return longest


    for j in range(len(s)):
        while s[j] in st: # check to see if j already exists in the set
            # if yes, remove from set and move i towards j till condition isn't met
            st.remove(s[i])
            i += 1

        window = (j - i) + 1
        longest = max(longest, window) # save the longer of the two values (current
longest value or window value)
        st.add(s[j]) # adds since it wasn't in the set



    return longest

print(lengthOfLongestSubstring('walmart')) # output 5 | 'lmart'
print(lengthOfLongestSubstring('Travis Wessman')) # output 9 | 'Travis We'


# Fixed Problem
def findMaxAverage(nums: List[int], k : int) -> float:

    curr_sum = 0
    max_sum = float('-inf') # can be anything greater than negative infinity
    i = 0 # left pointer

    for j in range(len(nums)):
        # expand window by adding right element
        curr_sum += nums[j]

        if (j - i + 1) > k: # if window size is bigger than k
            curr_sum -= nums[i] # remove the end
            i += 1 # increase by 1 index


        if (j - i + 1) == k: # when window size hits k, update answer
            max_sum = max(max_sum, curr_sum)

    return max_sum / k # keep track of max sum then average in retunr

ex1 = [1, 12, -5, -6, 50, 3]
print(findMaxAverage(ex1, 4)) # output 12.75 | Subarray 2: [12, -5, -6, 50] -> Sum = 51.
Average = 51 / 4 = 12.75

ex2 = [1, 3, 5, -7, 13, 23]
print(findMaxAverage(ex2, 3)) # output 9.666 | Subarray 3: [-7, 13, 23] -> Sum = 29.
Average = 29 / 3 = 9.666
```